

The project (no name except-study buddy- for the moment)

Core idea

A collaborative platform for university students to grow and professors to inspire and help.

This website will be designed for those lost students who can't access or find the answers for that one very specific question we all had at least once in our academic journey. It's for when the student/researcher gets failed by all their used-to-go platforms (youtube, AI, git-hub, stack-overflow.....), in short it will be a knowledge hub for peer-to-peer support and professor guidance, where users solve each other's problems and possibly learn new things in the process.

It might also serve as a support place and a showcase area where users share tips, ideas, plans of progress, road maps, or resources they find interesting -[only in their designated areas](#)-.....

What we realized in university is most of other majors don't get the support our major gets (even if not much related)....we get to have the AI and git-hub and stack overflow....as limited as they are, and as inaccurate as they can get, we still get the privilege most don't.

This platform might just solve that, because even if this idea was done before..it's not very popular for other majors.

So we make it as simple for the non tech-savvy people and as functional to be the very much help needed for all majors.. cause let's face it, the AI isn't good for everything

What we can attempt to make in short time and try the best to perfect is the first base of communication, try and make it Q&A functional at best we can -[with photo exchange support](#)-

First base of communication should make sure the user can post their questions/ideas as posts -[again, only in designated areas](#)-, while the response from other users would be as comments, these posts/comments will be judged by others with the vote method.

The vote would be 2 or 3 types for the comments:

Useful : the regular up vote for when those who read the comment find as helpful for op(Original Poster).

Useless : for when the comment is offensive, off-topic, or mockerybasically something that is simply useless.

Specialized : the higher form of useful and is given by the special members of the community or when the op themselves finds that comment exactly what they were hoping to find.

The usefull/useless votes will directly affect the user's account and ability of interacting with others on the platform through the ` rating system `....

The rating system: this is the counter of how many votes the user maintains through their posts/commentsthis might be just a simple counter display or will implement the 5 star rating type....the higher the useful counter the higher the rating gets and with it the more credibility that user will have thus others would find it easy to rely on their responses.. reaching a point of high(if we go with the 5 star type of rating ,we make it around 4stars),where is the users with that level reach ,comments on something, they will automatically get the "specialized " vote on their comment, they will also be able to give that vote to whom they think they deserve.

To keep the things balanced, each useless vote will deduct from the rating ,reducing the credibility of the user's comments and all.....

Other than the high credibility users, op can only give specialized vote to the comments of their own posts, non-more.

The post will simply obtain the useful /useless system...the higher the useful rate the more it will appear on other people's feed, the higher the useless rate, the lower the chance that post will appear.

One of the things I talked about before is the "designated areas".

What I mean by that is a special set of rooms.

5 main types of rooms to be precise, university room, field/major room, subject room, public space room, and the private rooms.

Self-explanatory really, each *university* will git its own *special room*,

Within each university there are majors, there for ..*major oriented room*...

For each major there are a set of *subjects* to study throughout the academic years...each with *their own room*..

The *public space room* is the free zone where the user can express ideas, share tips, or just be free with their thoughts and by that freedom be judged by other users through the rating system.

The *private rooms* are the by name suggests...private within the user who created it and who the user wants there...-[this might work with creating a passcode as google classroom does, or a user created password](#)-

That in my mind will give the users a clear specific oriented space to express their desires

Front end:

To build the base of the frontend we used react library in vite environment

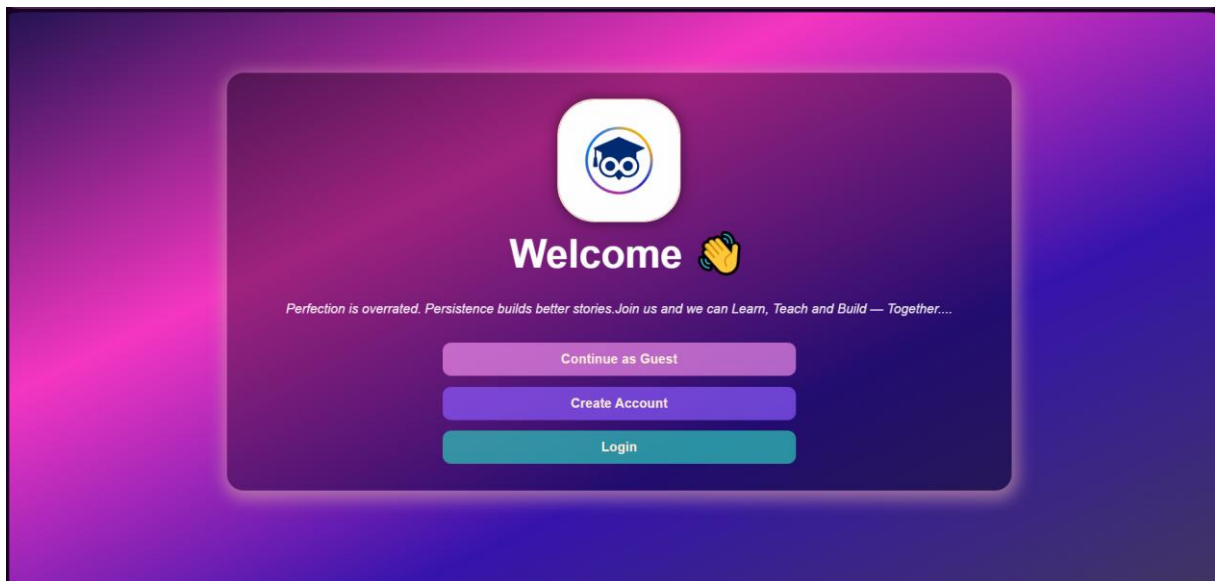
The project relies on core React features such as functional components, state management (useState), side-effect handling (useEffect), navigation (react 'navigate') and client-side routing to manage its interactive interface. Conditional rendering is used extensively to differentiate user roles, permissions, and visibility of features.

We made a simple vanilla elements pages and turned them into react....keeping external css files for each page...factoring most repetitive logic on its own and calling it later on where it was needed.....

To simulate a semi-real app behavior, we relied on local storage of the browser as a simple, ready to go database.

Some pages do not have so much done and other are done in simple minimalist way for the sake of keeping the heavy duty and logic to the backend.

1-welcome page:



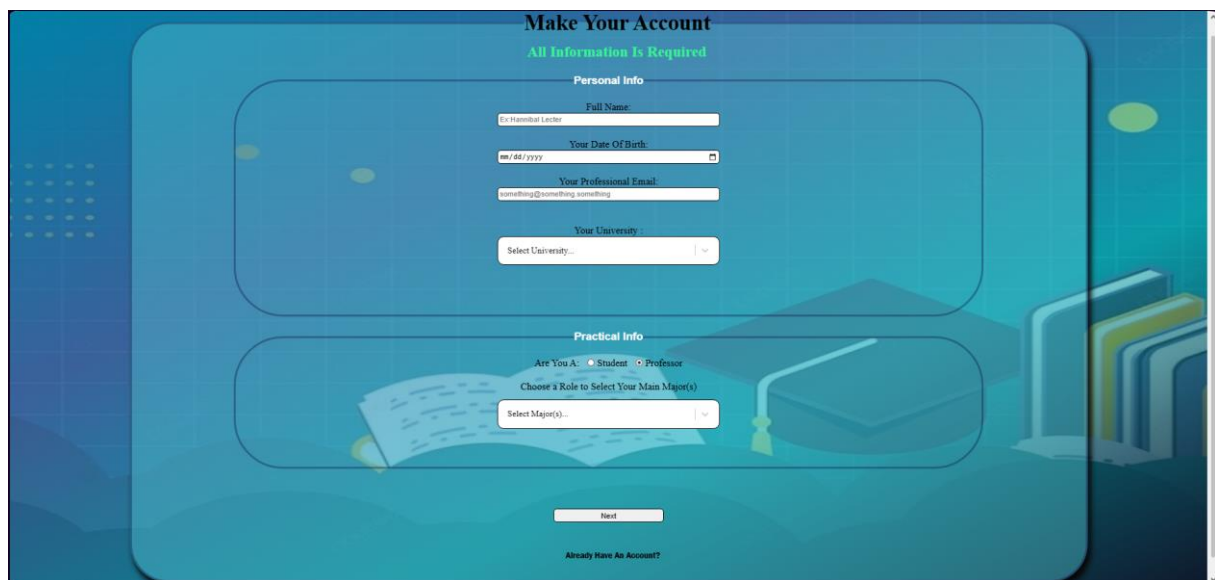
The first page the user sees when visiting the website, the only logic work done here is the routing to where each page should lead, only one to still need work is “continue as guest” button .

The guest idea:

The guests are the users who are not sure to create an account or not, so we give them a small peek on what the website looks like and how it functions in general.

what might be the final verdict on the guest logic Guests are restricted primarily to the public space and selected university rooms of their choice, which they can access in a read-only manner. A prompt allows guests to choose a limited number of university rooms to personalize their browsing experience. Around 3 to 5 maybe? there will be absolutely no saving of the session, once the browser tab is closed, everything the guest saw is gone....till their next visit Guests cannot post, comment, vote, or interact in ways that affect platform content, credibility, or user ratings. This preserves the integrity of the reputation system while allowing potential users to understand the platform's value before registering. This of course means that there will be constant feedback for every move they make.... Examples: "Log in to ask a question"////"Create an account to vote"////"Join to follow this discussion"///so on so forth...

2-information page:



The screenshot shows a web form titled "Make Your Account" with a subtitle "All Information Is Required". The form is divided into two main sections: "Personal Info" and "Practical Info".

Personal Info

- Full Name:
- Your Date Of Birth:
- Your Professional Email:
- Your University:

Practical Info

- Are You A: ☒ Student ☐ Professor
- Choose a Role to Select Your Main Major(s):

At the bottom of the form, there is a "Next" button and a link that says "Already Have An Account?".

For the sake of showing the entire form in one screenshot, this is 67% of its original size....moving on,

this basic personal information form, where the user is required to fill information from their name, date of birth, email for security reasons, their university (for the room).to their role as a student or a professor then their major..

one thing to note, we didn't use the simple html select element because
A-we couldn't provide all the selectable options so there has to be a dynamic filament implemented.

B-the traditional select is more of a hassle when it comes to multiple option select

C-the Select provided by the react library provides more flexibility and features and was quite ready to use.

for students: It's quite uncomplicated for each student would get to choose one specific major they are oriented in. thus this will add them automatically to the room of their major and the rooms of subjects related to that major.

For professors : the isMulti of the Select component provided by react is quite useful for multiple major selection, pair it with the role radio choice, only the professors can select more than 1 major.

Something that keeps bugging me is the professor role.....right now any student can falsely choose a professor role and continue with their registration....this should be dealt with somehow....should we add an input of some kind of proof? Or trust the users honestly when it comes to this decision....to be frank...the only difference I notice so far would be the tag next to the name and the major list selection....am I missing something? this should be on the consideration table for future plans , for this simply cannot stay like this.

Moving on to the logic implemented in the page,

On the basic information side, we implemented some validations so not anything can pass,...

The first name should atleast have 1 space and each word should atleast have 2 letters (done).

There is age restriction for the user should atleast be 17, a problem, we didn't do age limitations (people born in 1000 can pass, this should be fixed soon).

Email validation of the basic form [-something@something.something-\(done\). and](#) 2 accounts cant have the same email...because the recovering features rely on it.

A university selection list with the option to fill the one the user wants if messing from the list.

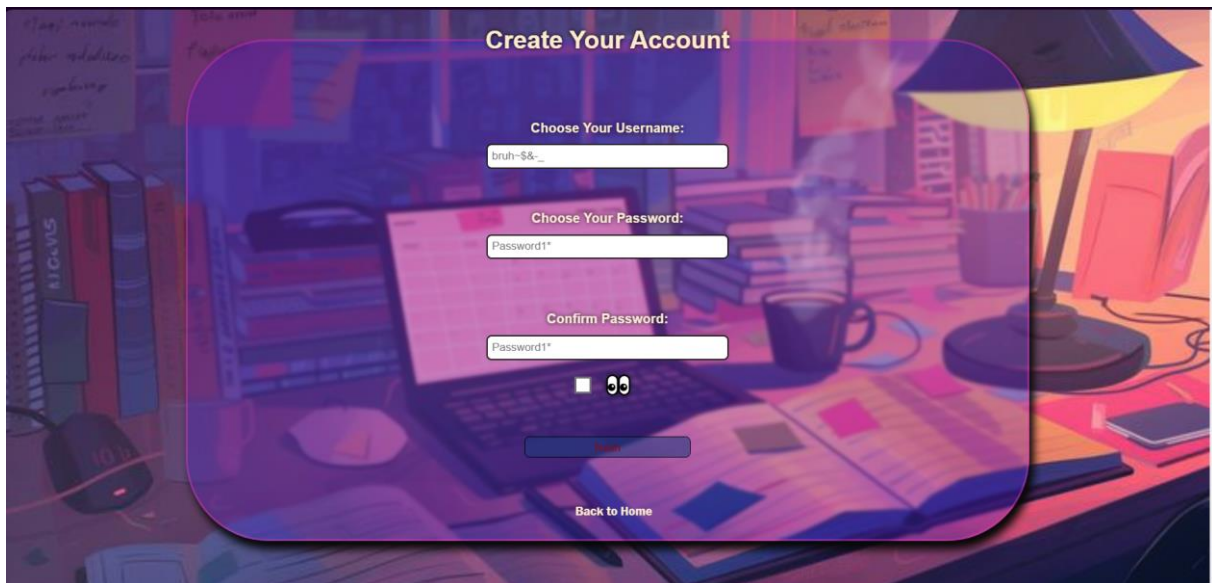
Practical information is based on role selection and major selection.

This is the place where the professor role can face some credibility and questioning.

The submitting of this form is only done when all the information are filled out...creating an initial profile of the shape:

```
const profile = {
  fullName,
  birthDate,
  email,
  university :finalUniversity,
  role,
  majors:finalMajors
};
```

3-register page:



this page is where the user chooses how they will display themselves in the platform, and how they will secure their accounts and privacy. yet this page has some regulations as not anything could pass, we used JavaScript to give in real time feed-back on the validation of the username and the strength of the password, how the user knows what they typed through “confirm password” a simple toggle to see it. And just to note, yet again the username is unique to their owner and cannot be repeated.

Like all parts of project, this is still work in progressthere is absolutely no way the passwords will be saved as they are (as we are testing in the local storage model) in the future backend and API, there will be hashing so the passwords will be one way encrypted for the security of user’s data.

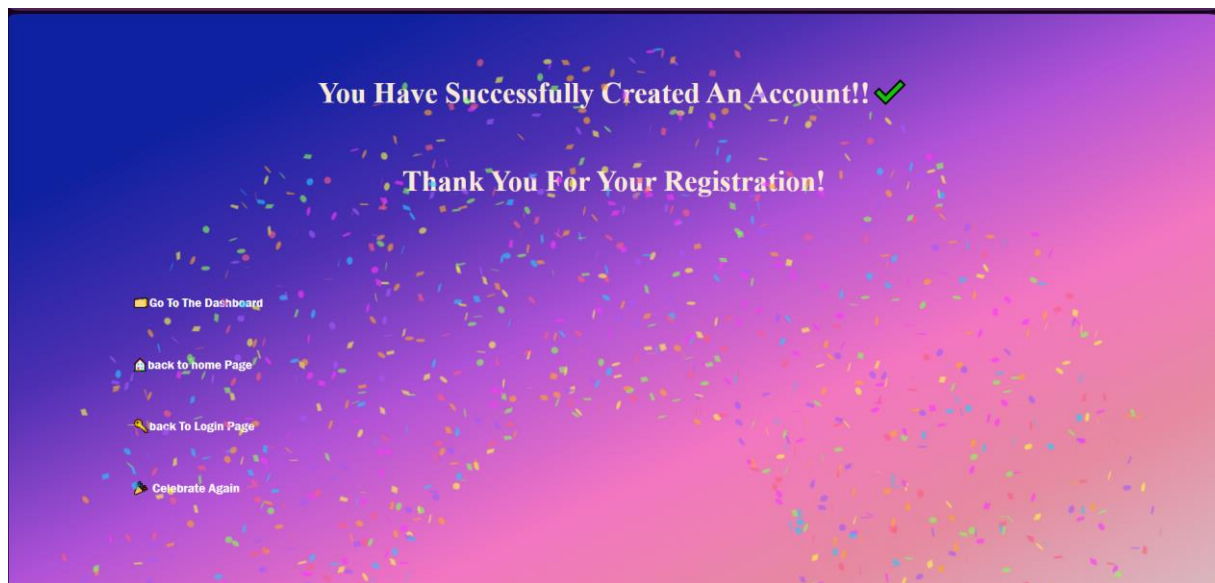
At the end and when the data is valid, this page will update the profile from info page to add the username and password.

So, the profile will be of 2 states:

```
hooks
  1 State: {birthDate: "2005-02-13", email: "allamnouh@gmail..."}
  2 State: {password: "Hithere1234", username: "noah3"}
```

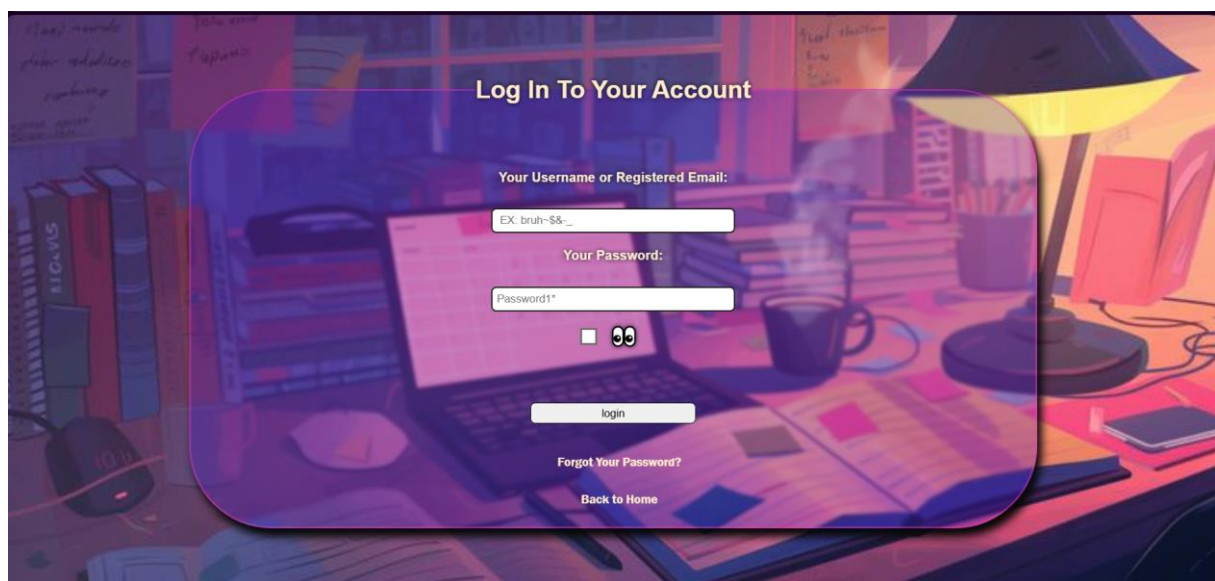
4-fin page

Other than ensuring the data is passing correctly among the pagesi don’t see what we can add to this page... It is a simple celebratory page, it has confetti, it has links to guide to where ever the user might possibly want to go, it even has a link behaving as a button to trigger the confetti..... I simply can’t see what else could be added here....



Again, grateful for the ready to use components in react.

5-login page:



Maintaining the same layout as the register page, we tried to keep this page as simple as it gets. Only checking is the data giving by the user actually exist in the database (in this case or level of production, in the local storage) .if so ,the user will access their account and thus continue to the platform from there, else simply throw error messages of credentials incorrect...

The login here will be on the form (username || registered email) && password.

Since there needs to be a password passing here....the backend will also implement the same hashing for the register page, but this time the API will only compare the hashed password...if matched to the email or username's provided password...that means login successful.....if not then it's an error to be handled

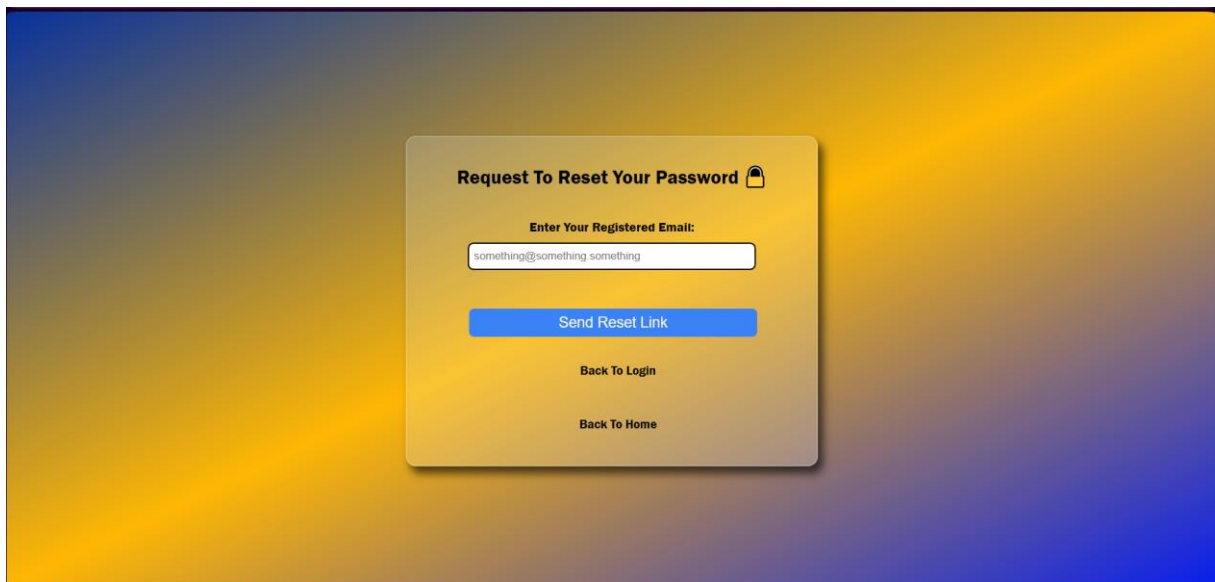
And this is the point where I again remind me and you about the error handling feedback....., at the login stage...the only 3 errors that could happen are mistakes at either input....simple message: "Invalid credentials" should be enough to deal with this... again. And if all errors are avoided, then login successful thus user gets directed to dashboard.

The logic done here is to pass the first input to the regex of email validation, if it passed then the system checks if it exists in the database(localstorage). If not, it will consider it as username and will check with database →if exists then check the password if not the error message.

in checking the password...since there is no hashing then it will check directly if exist...if no then error message, if yes, the login successful and navigate to the next page.

6-request reset password:

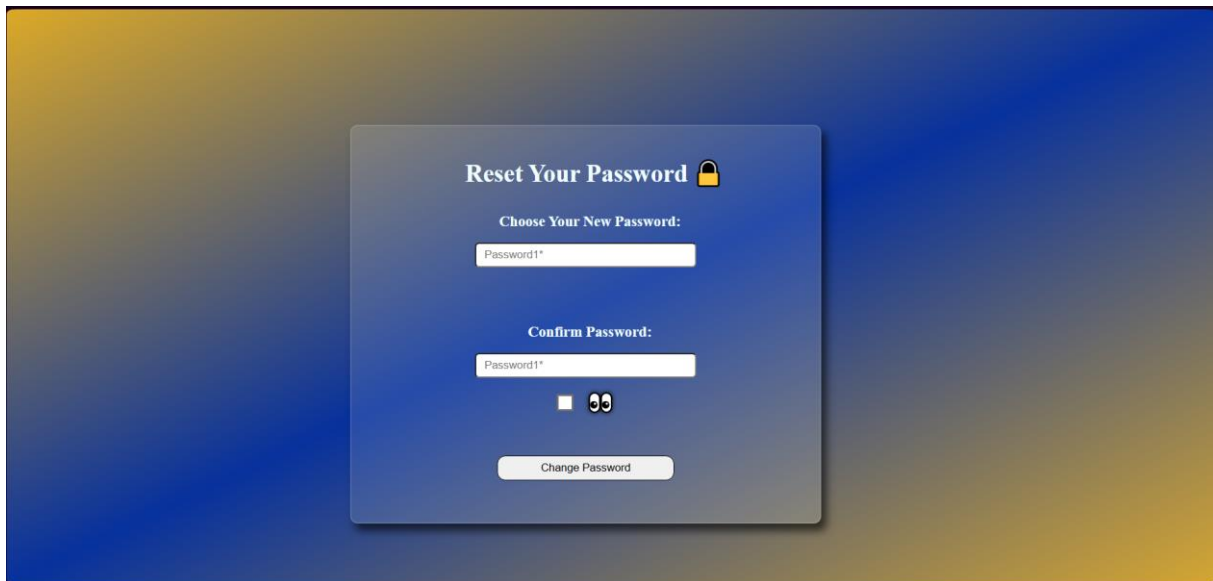
A link in the login page designed for the people who might forget their passwords...



Still no backend, when the system is full, this starts by validating the email format→check in the database if exists (this email is provided in the info form) →if it does then send an email to the said provided @ to have a link to the next page to reset the password.

For now, what we have, this will validate the email format, check if it exists in local storage, if it does then go to reset password page, if not then error message.

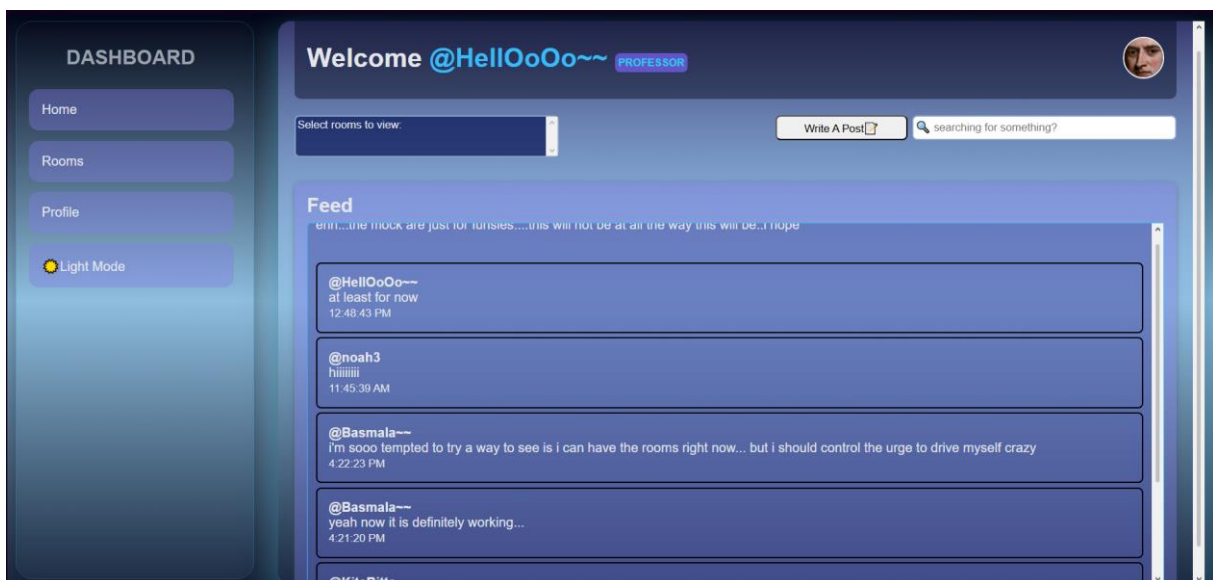
7-reset password:



the process is simple, change the password -same password confirmed- and this will directly update the database, so next time, the login will be with the new password.

As discussed on the register and login page, the same module of password hashing will be implemented ..this time to replace the existing password of the said account in the database. There will also be error handling as failing in sending the reset email link, the updating of the new password, the new password can't be the same as old the strength indicator of password the same as register...

8-dashboard:

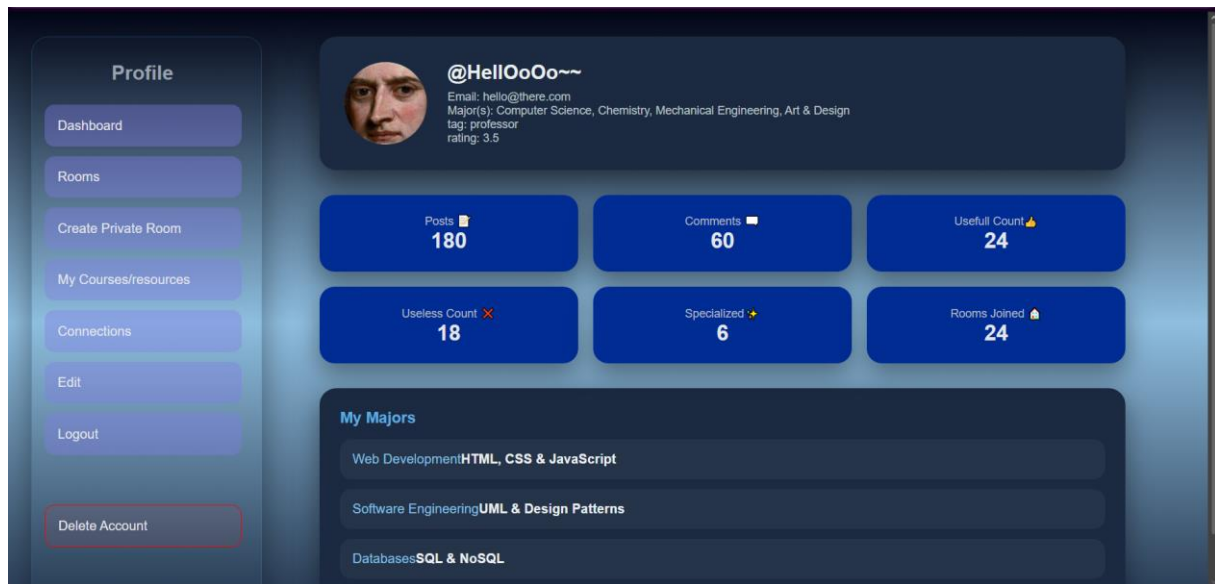


What is done so far is the display of almost entirely functional sidebar, a display of the username of the current user and their role tag. All using the same mock data from the previous pages stored in the local storage.

everything that is page would possibly have is almost entirely heavily reliant on the backend,

thus, this page still doesn't have much of a functionality.....

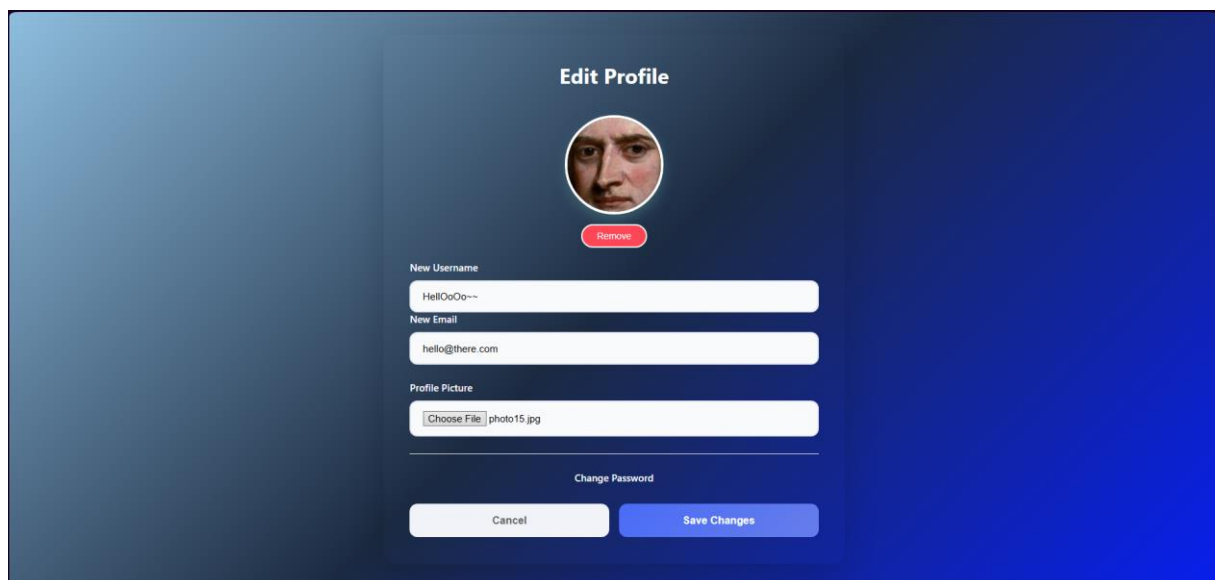
9-profile:



Display of the user's activity around the platform....most of these is still dead because this, as the dashboard is heavily reliant on the backend.

Same dynamic display of username, tag, majors, email.... placeholder for the rest.

10-edit page:



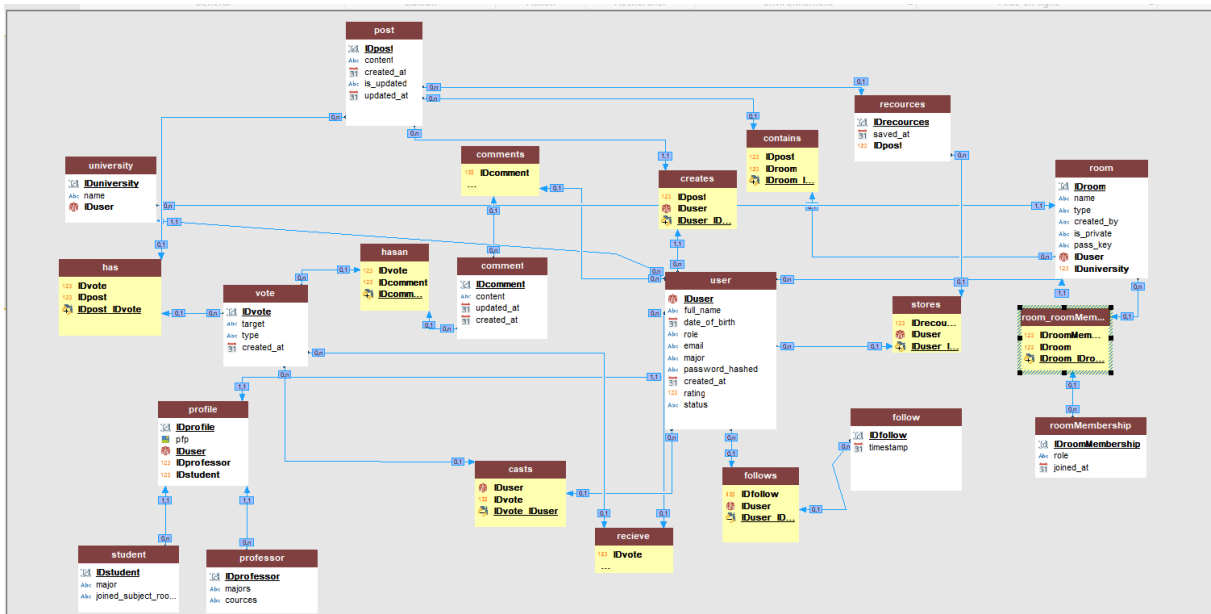
In our opinion, the only editable data should be the basic username, profile picture, email, and password. Other than data the rest of the data should remain untouched. This includes the rating and most of the information given in the first information form.

The link named "change password " leads to reset password page...this will later have some kind of mild confirmation since to access the edit page the user has to be logged in.

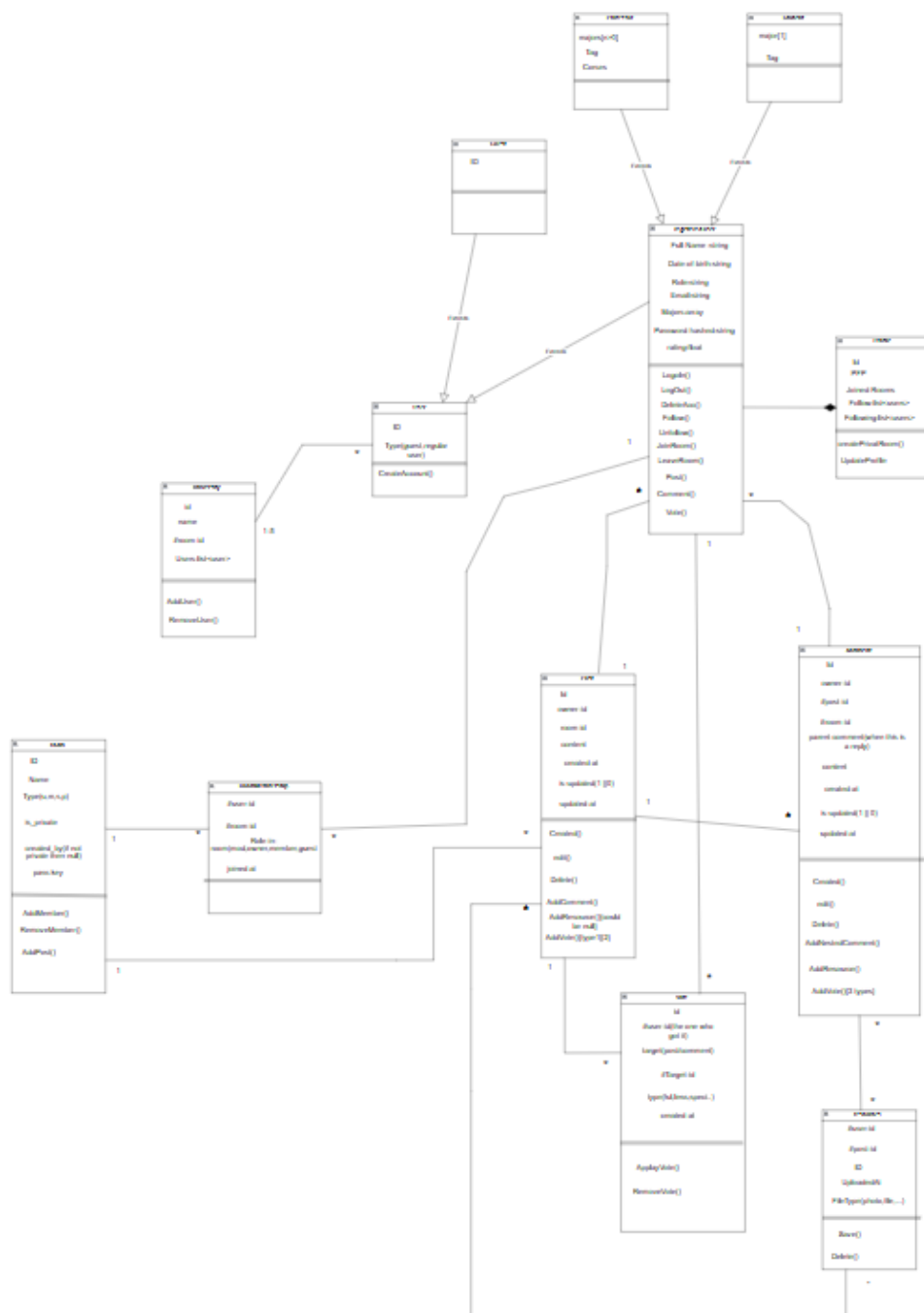
For now the confirmation is done by typing the username of the current active account.

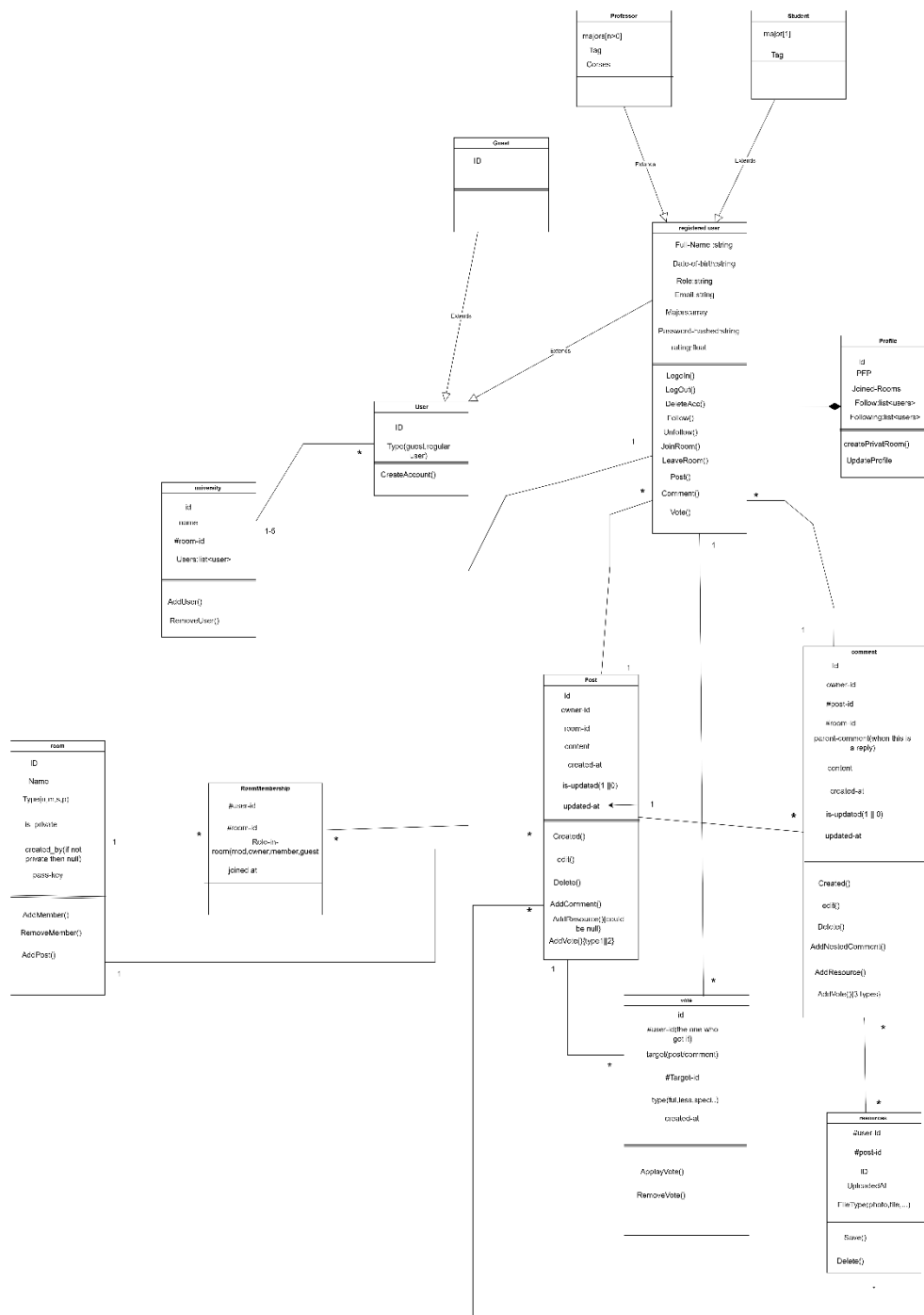
Primary diagrams

MCD:

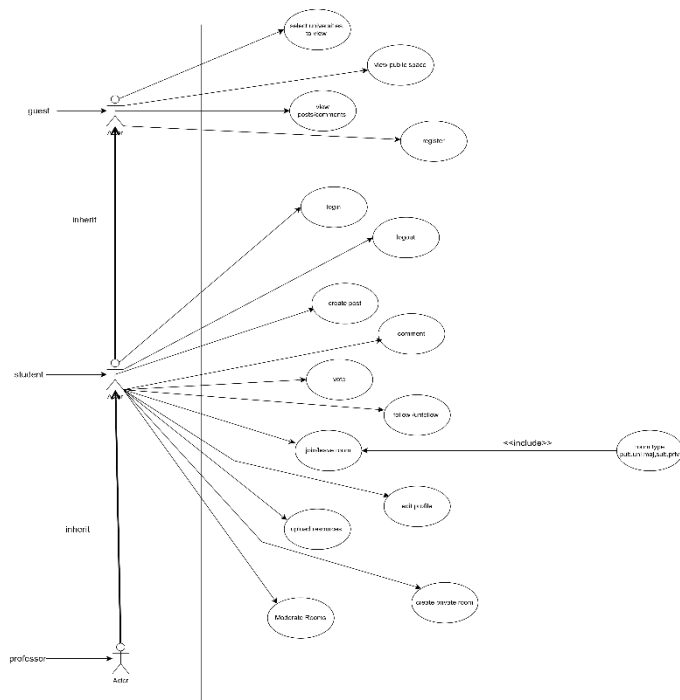


Class diagram:





Use case diagram:



The diagrams so far are still under work and study so they might be changed later.

Similar Platforms for Comparison

1. Piazza

- Pros: Highly organized for specific academic classes; direct interaction between students and instructors.
- Cons: Very closed-off; you can only see content for classes you are officially enrolled in.
- Our project: the project breaks these silos with Public Space rooms and University-wide rooms. Allow for "peer-to-peer support" across an entire institution, not just one classroom.

2. ResearchGate & Zenodo

- Pros: Excellent for hosting high-level research papers and formal datasets.
- Cons: Often "gate-kept" or overly formal; they focus on finished research rather than the messy "lost student" journey.
- our project: we focus on the "one very specific question" that standard research sites don't answer. the "Roadmaps" and "Plans of Progress" features offer practical guidance that formal research repositories lack+ the "will be provided" guest feature.

3. GitHub

- Pros: The gold standard for code collaboration and version control.

- Cons: Extremely technical and intimidating for "non-tech-savvy people". It is heavily biased toward Computer Science.
- Project : building for all majors. While GitHub is for code, the platform is a "Knowledge Hub" for everything from Art & Design to Civil Engineering. the UI is designed to be "simple and functional" for everyone.

4. Stack Overflow

- Pros: High-quality, vetted answers for technical questions.
- Cons: Infamous for being "unfriendly" to beginners; limited primarily to programming.
- Project : the Rating System is more encouraging, using a "Specialized" vote that can be granted by the original poster (OP) or highly-rated users, making it feel more like a community and less like an interrogation.

5. Reddit

- Pros: Massive, diverse communities for almost every topic.
- Cons: Information is "noisy" and easily lost in the feed; lack of formal academic credibility or professor presence.
- Project : the use of "Designated Areas" (specific rooms for majors/subjects) to keep discussions focused and organized. Unlike Reddit's general upvotes, the "Useful/Useless" system directly affects a user's Credibility Rating, which helps filter for quality academic advice.

6. Discord

- Pros: Great for real-time chat and casual community building.
- Cons: Terrible for "searchability"; answers to questions get buried in the chat history.
- Project : the project is Q&A function based with a structured post-and-comment layout. This ensures that once a "lost student" finds an answer, it remains findable and searchable for the next student, rather than disappearing in a scrolling chat log.(search feature in dashboard)

Extra notes....

As much as we danced around the idea of admin.....we ended up exactly needing it....

But as usual.. my ideas are never the regular type...

SuperAdmin: the ultimate admin with full control over EVERYTHING...

[validating profs, posts, comments, full control over the 4 public rooms-private rooms will still belong to the creator of them-, creating simpler admins, soon will discover if this is all or not]

Super admin can of course create simpler admins and control their level of power, + the superadmin is the only who gets to create and remove admin status. and they cant remove each other....superadmin stays as super admin....and they can behave like regular users....but with extra tag of "admin".

Admin :admin is a normal user with a set of duties and privileges....can moderate posts, comments, validate prof accounts and such...admin cannot create or remove admins....

To be an admin....the user have to be the good type of user....so....rating of 4 or more...basically the ones who got the privilege to give specialized vote.....

So those who got that rating will have the option to apply to admin position.....hold on...this also fixes the moderator problemthis will be by choice of the user rather then forced position. And the reporting problem....report to admin and admin reviews it and acts accordingly .

Back to the validation of prof....there is a proof required now...admins or superadmins will review that proof...if valid then valid prof, if not then reorientation as a student....if one major, oriented to that, if multiple, then the next time the user logs in they will be faced with a reorientations prompt to choose their main major much like anyother student....

Also in my first original documentation...(the papers I keep with me)I talked about "share"...then I thought to not do it....guess what we will be adding...the share option to posts...

Backend :

Backend will be with node.js as main environment and express.js as helper library, think of it as such:

Node.js **alone** can handle HTTP requests and responses. You could do everything manually using `http.createServer()`.

Express.js is a lightweight **helper library**. It makes routes, middleware, and request handling MUCH cleaner.

Node.js = engine of a car

Express.js = steering wheel, brakes, and gear shift

Now, we can do everything with just node.js...but I hated my self enough throughout the project that I want to make life a little simpler.

Since I never worked with backend before(album doesn't count)I'll be having baby steps roadmap that I will not follow but I think I should have either way....

Step 0 — Setup

- Node.js + Express.js
- CORS + JSON parsing middleware
- A single test route
- Frontend → Backend connection working

Goal: confirm the pipe works.

Step 1 — Store Users in Memory

- Make an array users = [] in backend
- On /register POST, push incoming profile into users
- Add default fields: isVerified: false, admin: false
- Return confirmation JSON to frontend

Goal: see that backend is **authoritative**; frontend sends data, backend stores it.

Step 2 — Validation(I don't think it was worth doing...)

- Move your existing validateProfile() logic to backend
- When a profile arrives, backend checks: full name, email format, role, majors, university
- If invalid → respond with error
- If valid → store and respond success

Goal: backend enforces rules, frontend just presents UI.

Step 3 — Uniqueness Checks

- Email uniqueness
- Username uniqueness
- Reject duplicates on backend, return message

- Frontend displays message

Goal: remove trust on frontend-only validation.

Step 4 — File Handling (Professor Proof +pfp updates)

- Accept a **proof document** via FormData in frontend
- Backend uses multer to process uploads 
- Save file path or store temporarily
- Associate it with user profile

Goal: full “professor registration” workflow.

Step 5 — In-Memory Role Logic

- Add admin and isVerified flags
- Superadmin account → hard-coded initially
- Optional: small route /promote to change admin status for testing

Goal: simple role system without database yet.

Step 6 — Optional: Database

- Move from memory array → PostgreSQL
- Each previous step works almost identically
- Only difference: data persists

Goal: scalable storage, real-world ready.

Step 7 — Authentication

- Register → store hashed password 
- Login → validate credentials 
- JWT token for sessions (optional, later) 
- Guest logic 

Goal: full backend authority, security layer.

Step 8 — Advanced Features

- Admin panel
- Role verification system
- Post moderation (this should come after posts but ok)
- Room system 

Goal: finish auth setup

Step 9 – Q&A logic

- Make post logic 
- Comment logic 
- The 2 type react for posts and 3 types for comments 
- Threaded comments 
- Option to save to resources 
- Option to share
- Option to report to admin

Step 10 – profile updates and history +search

- Populate the profile arias dynamically 
- Rating 
- Search function in dashboard returns profiles or posts 
- Clicking on the profile card lead to the card history

Might skip a few and connect directly to data baseI'm sick of false data mocks at this point.

All functionalities of this hellish platform that the admin and superadmin should be included in :

- Validate the other option from the majo and uni "other" input.
- Accept or refuse the requests of creating subject rooms (the option to request them will be added later)
- Look into reports on private rooms
- Moderate the public rooms (ps,u,m,s)
- Moderate posts and comments
- Verify professor's provided proof of status.
- See reports on posts and comments and deal with them

- Suspend user
- Approve link and file resources.
- Delete inactive users account
- Delete users account if much violation happened.

Superadmin:

- Everything the admin can do
- Upgrade user to admin(give specific permissions)
- Remove admin title.
- Add superadmins.
- See platform logs and such(idk about this one)

me and cluade reached an agreement:

Admin: *  Validate "other" university/major inputs *  Accept/refuse subject room creation requests *  Look into reports on private rooms *  Moderate public rooms (subject/uni/public/major) *  Moderate posts and comments (edit/delete/hide) *  Verify professor proof *  See and handle reports on posts/comments *  Suspend users (temporary ban) *  Approve resource links and files in posts *  Delete inactive accounts *  View all users — search, filter by role, rating, status *  Unsuspend users — lift a suspension early *  Reject professor status — demote back to student with a reason and reorientation to one major (if they registered with more then 1 major/the choice is for the user to select the major to be oriented to) SuperAdmin: *  Everything admin can do *  Upgrade user to admin with specific permissions *  Remove admin title *  Add other superadmins *  See platform logs *  Delete accounts with repeated violations *  Manage admin permissions individually — one admin can moderate posts but not suspend users for example *  Override any admin action — reverse a suspension, restore a deleted post *  Platform-wide announcements — post a message that appears for all users *  View platform stats — total users, posts, reports, active rooms *  View violation history per user — how many reports, what actions taken * access all rooms without any exception

the option to report private rooms might be extra(just like share posts)but we will see later

the rest so far(from what I see)

Admin Panel:

- ☐ Validate "other" university/major inputs — users who registered with custom uni/major need a review queue
- ☐ Subject room creation requests — the request flow doesn't exist yet (users can't request rooms, admins can't approve them)
- ☐ Resource approval — flagged posts with links/files need a review queue tab
- ☐ Violation history per user — clicking a user should show their actionHistory
- ☐ Hidden content management — see what's been hidden, restore if needed

SuperAdmin Panel:

- ☐ Override admin actions — restore hidden posts, lift suspensions from the logs
- ☐ Platform-wide announcements display on dashboard — announcements exist but dashboard doesn't show them yet

Professor validation flow:

- ☐ Proof upload at registration — professors need to upload a document during registration, currently there's no file input for that
- ☐ proofFile path stored on user — without the upload the verification tab in admin is empty

Reporting system:

- ☐ Report button on posts and comments — users can't actually report anything yet, the reports array on posts/comments is always empty
- ☐ Private room reporting

Moderation:

- ☐ Auto-hide content after X useless votes threshold
- ☐ Suspended user login block — suspended users can still log in right now

Academic & Data Validation

validate_university

validate_major

Handles:

- “Other” input validation
- Keeping academic data clean

Room Governance

approve_room_creation
reject_room_creation
delete_room
manage_room_roles
moderate_public_rooms
moderate_private_rooms
view_room_reports

Covers:

- Room creation flow
- Public vs private moderation
- Membership control

Content Moderation

delete_post
delete_comment
edit_post
edit_comment
view_content_reports
handle_content_reports

Covers:

- Posts
- Comments
- Report handling

User Management

suspend_user
unsuspend_user
delete_user
view_user_profile_admin

Covers:

- Ban / unban

- Account deletion
- Admin-level profile access

Professor Verification

verify_professor

reject_professor

Resource Moderation

approve_resource

reject_resource

Controls:

- Links
- PDFs
- Uploaded content

Reports System

view_reports

resolve_reports

Generic reporting system (posts, comments, rooms, users)

Maintenance

delete_inactive_users

Platform cleanup

Superadmin Exclusive Actions:

promote_to_admin

demote_admin

assign_permissions

create_superadmin

view_platform_logs

override_any_action

